

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Dr. David Rohr and Anran Zhang

Exercise 2: Kinematics

Session date: 03/03/2026

Task 1: 3R Robot Kinematics

For the 3R manipulator shown in Figure 1, solve the following tasks:

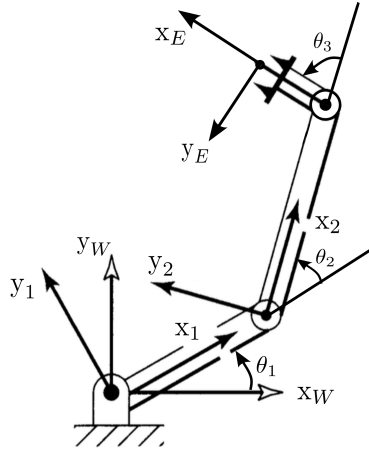


Figure 1: Planar 3R manipulator with parallel joint axes. The configuration is defined by joint angles $\theta_1, \theta_2, \theta_3$ and link lengths l_1, l_2, l_3 . A world frame $\underline{\mathcal{F}}_W$ is attached at the base. An end-effector frame $\underline{\mathcal{F}}_E$ is attached to the gripper tip point as shown.

i) Let, for notational brevity, $[t_x, t_y]^T := {}_W\mathbf{t}_{WE}$ and ϕ_E the orientation of the end-effector (angle from x_W to x_E). Compute the forward kinematics, i.e., the end-effector pose $[t_x, t_y, \phi_E]^T$ expressed in $\underline{\mathcal{F}}_W$ as a function of the joint states $\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3]^T$.

ii) The Jacobian $\mathbf{J}(\boldsymbol{\theta})$ of the manipulator expressed in $\underline{\mathcal{F}}_W$ is defined as

$$\mathbf{J} := \partial[t_x, t_y, \phi_E]^T / \partial \boldsymbol{\theta}$$

and it can be used to map between joint velocities and end-effector twist as

$$[\dot{t}_x, \dot{t}_y, \dot{\phi}_E]^T = \mathbf{J}(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}$$

Derive the analytical expression for $\mathbf{J}(\boldsymbol{\theta})$.

iii) Determine the singular configurations of the manipulator, i.e. $\boldsymbol{\theta}$ s.t. $\det(\mathbf{J}(\boldsymbol{\theta})) = 0$. Can you provide an intuitive explanation of the singularities and determine which degrees of freedom are lost at these configurations?

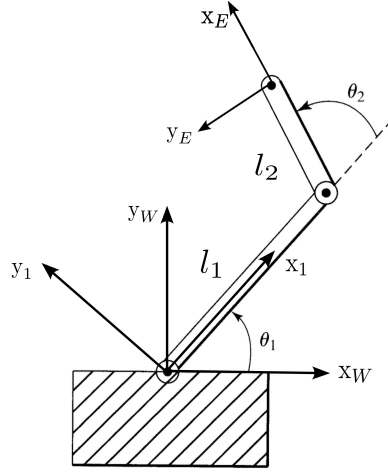


Figure 2: 2R Robot configured using joint parameters θ_1, θ_2 and link lengths l_1, l_2 .

Task 2 (Advanced): Isotropic Points

Besides singularities, a manipulator may exhibit other special configurations, such as isotropic configurations. These are characterized by the Jacobian matrix having orthogonal columns of equal norm, i.e., $\mathbf{J}^T \mathbf{J} = \delta \mathbf{I}$, $\delta > 0$. Consider the 2R manipulator shown in Figure 2. Its Jacobian in end-effector frame $\mathcal{F}_E$ (with respect to gripper position only, ignoring orientation) is given as:

$$\mathbf{J}(\boldsymbol{\theta}) = \begin{pmatrix} l_1 \sin \theta_2 & 0 \\ l_1 \cos \theta_2 + l_2 & l_2 \end{pmatrix}$$

- i) Determine the manipulator's isotropic points (treat both θ_i and l_i as free variables) and draw the manipulator in the corresponding configuration(s).
- ii) Provide an intuitive interpretation of the special role that the isotropic configuration plays.

Task 3 (Advanced): Kinematics Computation in Python

In this task, you will implement forward kinematics (FK), Jacobian computation, and inverse kinematics (IK) for a mobile manipulator (Tiago) using a modern kinematics library (e.g., Pinocchio).

By completing this task, you should be able to:

- Understand the robot kinematic model derived from a Universal Robot Description File (URDF).
- Compute forward kinematics and extract end-effector poses.
- Compute the geometric (position) Jacobian of the end-effector.
- Implement a numerical inverse kinematics (IK) solver.

You will begin with model inspection and visualization, then move to Jacobian-based kinematics computation, and finally implement a numerical inverse kinematics solver. Follow the step-by-step instructions provided in the Jupyter notebook.

i) Model Loading and Visualization

- Load the robot model from its URDF file.
- Inspect the kinematic structure (joints, frames, DoFs).

- Visualize the robot home configuration.

ii) Numerical Inverse Kinematics

The mapping from joint states to end-effector pose is often non-linear and an analytical inverse may not exist (cf. overactuated systems, i.e. if the degrees of freedom provided by the joints are larger than the end-effector DoFs. Think of yourself being able to reach for an object with one hand while still maintaining a lot of freedom in how exactly you pose your body for this task).

Nevertheless, we can solve for joint states given a desired end-effector pose using an iterative solving scheme. Given a desired end-effector position¹ ${}^W\mathbf{t}_{WE,\text{goal}}$, we leverage the associated Jacobian $\mathbf{J}(\boldsymbol{\theta})$: $d({}^W\mathbf{t}_{WE})/dt = \mathbf{J}(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$ – which is easy to obtain and (pseudo) invert – for relating small increments Δ in end-effector position $\Delta({}^W\mathbf{t}_{WE})$ and, correspondingly, the individual joints states $\Delta\boldsymbol{\theta}$. Using the Jacobian as linearization of the nonlinear forward kinematics $\Delta({}^W\mathbf{t}_{WE}) \approx \mathbf{J}(\boldsymbol{\theta})\Delta\boldsymbol{\theta}$, we can solve for $\Delta\boldsymbol{\theta}$ using the pseudo-inverse $\mathbf{J}^\dagger(\boldsymbol{\theta})$

$$\Delta\boldsymbol{\theta} = \mathbf{J}(\boldsymbol{\theta})^\dagger \Delta({}^W\mathbf{t}_{WE})$$

with

$$\mathbf{J}(\boldsymbol{\theta})^\dagger = \mathbf{J}(\boldsymbol{\theta})^T (\mathbf{J}(\boldsymbol{\theta})\mathbf{J}(\boldsymbol{\theta})^T)^{-1}$$

and

$$\Delta({}^W\mathbf{t}_{WE}) = k_p({}^W\mathbf{t}_{WE,\text{goal}} - {}^W\mathbf{t}_{WE})$$

designed to take incremental steps towards the goal. Update

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$$

and iterate until the position error norm falls below a predefined threshold.

- $\mathbf{J}$ is the end-effector Jacobian,
- $\mathbf{J}^\dagger$ is its Moore–Penrose pseudo-inverse,
- k_p is a proportional gain,
- $\Delta\boldsymbol{\theta}$ is the joint increment.

¹Analog reasoning for orientation. For the scope of this exercise, however, we consider only the position.